ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

CENG 407

Software Requirements Specification

TARAS: Agricultural Analysis & Simulation

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

TABLE OF CONTENTS

1. Introduction....2
1.1. Purpose....2
1.2. Scope....2
2. General Description....2
2.1. Glossary....2
2.2. User Characteristics....5
2.2.1. Primary User: Farmer....6
2.2.2. External Party: Stakeholders....6
2.3. Overview of Functional Requirements....7
2.4. General Constraints and Assumptions....7
2.4.1. Constraints....7
2.4.2. Assumptions....9
3. Specific Requirements....10
3.1. Interface Requirements....10
3.1.1. Hardware Interface Requirements....10
3.1.2. Communication Requirements....11
3.1.3. System and Backend Requirements....12
3.1.4. Software Interface Requirements....13
3.1.5. User Interface Requirements....14
3.2. Detailed Description of Functional Requirements....15
3.3. Data Preprocessing....30
3.3.1. Irrigation Data Preprocessing....30
3.3.2. Image Data Preprocessing for Disease Detection....32
3.4. Non-Functional Requirements....33
3.4.1. Performance Requirements....33
3.4.2. Reliability Requirements....34
3.4.3. Availability Requirements....34
3.4.4. Scalability Requirements....34
3.4.5. Security Requirements....35
3.4.6. Usability Requirements....35
3.4.7. Maintainability Requirements....35
3.4.8. Portability Requirements....36
4. Analysis - UML....36
4.1. Use Cases....36
4.1.1. Use Case Diagram....36
4.2. Functional Modelling(DFD)....52
4.2.1. DFD Diagrams....52
5. Conclusion....53
6. References....54

1. Introduction

1.1. Purpose

The purpose of this Software Requirements Specification is to define the objectives and functional boundaries of the TARAS system. TARAS is designed to support farmers in making accurate, data-driven decisions about irrigation, plant health and overall field management. The system integrates distributed IoT sensors to monitor soil and environmental conditions, applies ML models to generate zone-specific irrigation recommendations and incorporates computer-vision-based disease assessment to help with the early detection and prevention of crop loss.

This document specifies the requirements for the TARAS mobile application, which provides the primary user interface for real-time field visualization through a digital twin, personalized notifications, decision support and sustainability tools such as carbon-footprint estimation. The purpose of this SRS is to define the software's goals, intended functions, and constraints so that all stakeholders share a common understanding of the system to be developed.

1.2. Scope

The scope of this project includes developing an end-to-end precision-agriculture system that collects soil and environmental data through IoT sensor nodes and transmits them directly to a cloud backend via Wi-Fi. The collected data are stored in a cloud database and processed using Real-world based algorithmic calculations for irrigation prediction, computer-vision algorithms for disease detection and a large language model for decision support. A mobile app provides farmers with digital-twin visualizations of soil moisture levels, real-time alerts, recommendations and carbon-footprint estimation.

2. General Description

2.1. Glossary

Term	**Definition**
Farmer	The primary user of TARAS who manages the agricultural field, monitors irrigation, and responds to alerts using the mobile application.
Stakeholder	An organization or representative entity such as an agricultural cooperative, farmer association, or advisory body that uses aggregated and anonymized TARAS outputs for monitoring field performance, supporting farmers, and improving collective decision-making.

Digital Twin	A virtual representation of the physical agricultural field that integrates real-time sensor data, spatial information, and analytical models to simulate field conditions and support decision-making.
Field	The physical agricultural land managed by the farmer, where crops are cultivated and monitored using TARAS.
Field Zone	A distributed embedded device deployed in the field that collects data and transmits it wirelessly to the gateway.
IoT Sensor Node	A central device that aggregates data from multiple IoT sensor nodes and forwards it to the backend system via the internet.
ESP32	A low-power microcontroller with integrated Wi-Fi and Bluetooth capabilities, used as the main controller in IoT sensor nodes for data acquisition and communication.
SHT	The name of the series of digital temperature and humidity sensor products that provides high-accuracy environmental measurements via an I²C communication interface.
Capacitive Soil Moisture Sensor	A sensor that measures soil moisture levels by detecting changes in capacitance caused by water content in the soil, offering improved durability over resistive sensors.
Telemetry	The automated process of collecting measurement data from remote sensors and transmitting it to a central system for monitoring and analysis.
Time-Series Data	Data collected sequentially over time intervals, such as sensor readings, used to analyze trends, patterns, and temporal changes in field conditions.
Irrigation Recommendation Engine	An algorithm based component that analyzes sensor data, historical records, and crop requirements to generate optimized irrigation suggestions.
Disease Detection Module	A computer-vision component that analyzes leaf images using CNN models to identify plant diseases, estimate severity, and provide early alerts to the farmer.
LLM-based Advisory System	A decision-support component that uses a Large Language Model to generate natural-language guidance for farmers by interpreting sensor data, irrigation recommendations, and disease detection results.
Carbon Footprint	A calculated metric representing the estimated greenhouse gas emissions associated with agricultural activities such as irrigation, fertilizer use, and energy consumption.
REST API	A stateless application programming interface that enables

	communication between system components using standard HTTP methods and structured data formats such as JSON.
Backend	The server-side components of the TARAS system responsible for data storage, processing, analytics, and integration with machine-learning models.
Frontend	The user-facing components of the TARAS system, including the mobile application interface through which users view data, receive alerts, and interact with recommendations.
PostgreSQL	An open-source, ACID-compliant relational database management system that supports advanced features such as JSONB, extensions, strong indexing, and complex queries, making it suitable for scalable and data-intensive applications.
TimescaleDB	A PostgreSQL extension optimized for time-series data. It uses hypertables to efficiently store, query, and compress IoT sensor readings.
PostGIS	A spatial database extension for PostgreSQL that adds support for geographic objects, enabling storage and analysis of field zones, polygons, sensor locations, and other geospatial data used in the digital twin of the farm.
CNN(Convolutional Neural Network)	A deep-learning model designed for image analysis, used in this project to detect plant diseases by classifying leaf images and identifying visual patterns of stress or infection.
ML Model	A trained computational model that learns patterns from data and generates predictions or classifications, such as irrigation needs or disease likelihood.
Inference	The process of applying a trained machine-learning model to new input data to generate predictions or outputs.
Store-and-Forward	A communication strategy in which data is temporarily stored locally when connectivity is unavailable and transmitted once the connection is restored.
Deep Sleep Mode	A low-power operating state of microcontrollers in which most system functions are disabled to conserve energy between data collection cycles.
JWT(JSON Web Token)	A secure, compact token format used for authentication and authorization between the frontend and backend components of the system.
Telemetry Packet	A structured data unit containing sensor readings, timestamps, and device identifiers transmitted from sensor nodes to the gateway or backend.

Threshold	A predefined limit or value that, when exceeded or violated, triggers system alerts or recommendations.
Normal Operating Conditions	Conditions in which the system operates with stable hardware, expected network connectivity, and no major external disruptions such as prolonged power outages or hardware failures.
Guidance Session	A short onboarding or instructional interaction designed to help users understand basic system functionality.
Evapotranspiration(ET_0)	ET_0 is a standardized measure of atmospheric water demand, representing the evapotranspiration rate from a hypothetical reference surface under given meteorological conditions. It reflects the effect of weather conditions on water loss independently of crop type.
Crop Coefficients(Kc)	Kc is a dimensionless coefficient used to adjust reference evapotranspiration (ET_0) to account for crop type and growth-stage characteristics, enabling estimation of crop-specific water demand.
ACID	A set of four fundamental properties—Atomicity, Consistency, Isolation, and Durability—that ensure database transactions are processed reliably and maintain data integrity, even in the presence of errors or system failures.
I^2C	A synchronous serial communication protocol that uses two lines (Serial Data Line (SDA) and Serial Clock Line(SCL)) to connect a microcontroller with multiple peripheral devices, enabling simple, low-power data exchange over short distances in embedded systems.
ADC	A hardware component that converts continuous analog signals (such as voltage levels from a soil-moisture sensor) into discrete digital values that can be processed by a microcontroller.
DAP	A time-based index that represents the number of days elapsed since a crop was planted, commonly used in agriculture to track crop growth stages, schedule irrigation or fertilization, and map crop coefficient (Kc) profiles to specific development periods.

2.2. User Characteristics

The system is designed with two key user groups in mind: the primary user (farmer), and external institutional stakeholders (such as cooperatives or environmental reporting bodies). Each group has distinct abilities, responsibilities and interaction needs, which shape the system's interface and functionality.

2.2.1. Primary User: Farmer

Profile: Small-to-medium-scale farmers who rely on their experience for regular irrigation and crop-health decisions. They interact with the system mostly in outdoor field conditions.

Age Range: Typically 20–65, though many may be older and accustomed to traditional farming practices.

Technical Skills: Low-to-moderate comfort with digital tools. Interfaces shall be simple, intuitive and optimized for mobile use, with minimal steps and clear visual cues.

Work Environment: Outdoor, field-based conditions with limited connectivity and variable lighting.

Key Characteristics:

- Relies heavily on intuitive decision-making
- Faces challenges like water scarcity, plant disease uncertainty
- Needs clear, non-technical explanations rather than complex analytics
- Requires solutions that are low-cost, reliable and field-tolerant

2.2.2. External Party: Stakeholders

Profile: Stakeholders are organizations that have access to the system but with highly restricted, read-only permissions. They do not participate in daily decision-making or sensor management. Their role is limited to viewing summaries and sustainability-related metrics for monitoring or reporting purposes.

Examples:

- Agricultural cooperatives
- Sustainability and environmental monitoring institutions
- Government programs that track water usage, efficiency, or carbon impact

Interaction Model: Stakeholders log into a summary-focused dashboard that displays high-level insights such as water consumption trends, irrigation efficiency metrics and carbon-footprint estimates. They don't upload data, adjust system settings or interact with field-level features such as disease detection or digital-twin visualizations. Their interaction is passive and observational.

2.3. Overview of Functional Requirements

TARAS provides an integrated set of functional capabilities to support data-driven decision-making in smart agriculture. At its core, the system continuously collects soil and environmental data from distributed IoT sensor nodes deployed in the field. These data are transmitted directly to the cloud backend via Wi-Fi connectivity, where they are stored as time-series records and made available for analysis, visualization, and decision-support processes.

Based on the collected data, TARAS performs analytical and decision-support functions, including machine-learning–based irrigation recommendations and computer-vision–based plant disease detection.

The mobile app serves as the primary interaction tool for all users to visualize real-time and historical field conditions through a digital twin, receive recommendations, upload plant images and interact with advisory features. In addition, the system supports sustainability-related functions such as carbon footprint calculation when farmers input necessary information.

Together, these functional requirements define an end-to-end system that integrates sensing, communication, data processing, analytics, visualization and user interaction to deliver reliable and actionable agricultural decision support.

2.4. General Constraints and Assumptions

2.4.1. Constraints

- Hardware Constraints:

  The sensor nodes operate on ESP32 microcontrollers with limited RAM and processing capability, which restricts complex on-device computation. As a result, most data processing, analytics, and decision-making tasks are offloaded to cloud-based services accessed directly via Wi-Fi connectivity.

  All soil-moisture, temperature, and humidity sensors must operate within strict voltage ranges because of the ESP32's output limitations (typically 3.3 V), limiting hardware choices[1]. Since all sensor nodes are battery-powered, energy consumption must remain minimal to ensure long-term field deployment.

- Communication Constraints:

  The system relies on Wi-Fi connectivity for direct data transmission from ESP32 sensor nodes to the cloud

backend. Although Wi-Fi offers higher bandwidth compared to low-power long-range protocols, its effective range and reliability can be limited in rural agricultural environments. Variations in signal strength and intermittent internet access may lead to temporary data transmission failures, requiring retry mechanisms and buffering strategies at the node level to ensure reliable synchronization with the cloud services.

- Environmental Constraints:

  Devices that are all field-deployed must withstand harsh outdoor conditions such as humidity, temperature extremes, rainfall, dust, and direct sunlight. Variations in soil composition and microclimatic differences introduce noise into sensor readings, potentially affecting predictive model accuracy.

- Software & Data Constraints:

  Available training data for machine-learning models is limited and may not represent all crop types or environmental contexts, reducing early-stage model accuracy[2].

  Real-time inference must meet latency requirements appropriate for on-field decision-making, which imposes limits on model size and computational complexity.

- Operational Constraints:

  Continuous internet connectivity cannot be guaranteed for either the gateway or the user's mobile device, requiring offline-tolerant system behavior. The mobile application must support a wide range of user devices, specifically Android versions 7.0–16.0 and iOS versions 15.1–26.1.

- Budget & Resource Constraints:

  High-quality agricultural sensors and reliable power-management components represent a significant portion of the system cost. In addition, ensuring continuous Wi-Fi or internet connectivity in agricultural environments may require auxiliary networking equipment, such as mobile hotspots or cellular modems, which can increase deployment and operational expenses.

  Real-world field access for testing may be limited, which may reduce the frequency and duration of validation cycles.

2.4.2. Assumptions

- Sensor Deployment Assumptions:

  Sensors are assumed to be properly installed, calibrated, and positioned in pre-determined areas of the field. Battery-powered sensor nodes are assumed to maintain sufficient charge throughout their scheduled data collection intervals.

- Communication Assumptions:

  The sensor nodes are assumed to operate within the coverage range of a stable Wi-Fi network or an alternative internet access point. Temporary connectivity outages are expected in agricultural environments; however, it is assumed that retry and buffering mechanisms will allow buffered data to be transmitted successfully once connectivity is restored, ensuring eventual synchronization with the cloud backend without permanent data loss.

- User Behavior Assumptions:

  Farmers are assumed to regularly monitor the mobile application and take appropriate actions based on irrigation recommendations or disease analysis results. External stakeholders are assumed to require only high-level, read-only access for monitoring and reporting purposes.

- System Assumptions (Irrigation & Disease Detection):

  Soil-moisture, temperature, and humidity sensor readings are assumed to accurately represent current field conditions for irrigation decision-making.
  The rule-based irrigation logic assumes that moisture trends and threshold crossings reliably indicate crop water stress under normal operating conditions.

  Images uploaded for disease detection are assumed to be clear, correctly captured, and representative of the plant's current health state.

  Environmental anomalies or abnormal conditions (e.g., extreme weather, sensor faults, or equipment malfunctions) are assumed to be detectable through irregular sensor behavior or inconsistent image analysis results.

  Calibration parameters and decision rules are assumed to remain valid for the defined crop type and field conditions unless manually updated.

- Mobile Application Assumptions:
  The mobile application assumes intermittent but recurring Internet connectivity to retrieve sensor data, digital twin updates, and ML-driven recommendations.

3. Specific Requirements

3.1. Interface Requirements

3.1.1. Hardware Interface Requirements

The TARAS system interacts with a distributed network of IoT sensor nodes that communicate directly with cloud-based services via Wi-Fi connectivity, requiring consistent electrical, communicational and operational compatibility across all hardware components. Each sensor node incorporates an ESP32 microcontroller, which provides the necessary GPIO pins, I²C bus capabilities and low-power operational modes required for continuous monitoring. The hardware interface must support stable connections to capacitive soil-moisture sensors, SHT31 temperature-humidity units and additional digital sensors through standardized I²C communication. All sensors must operate within the microcontroller's voltage range (3.0–3.6 V) and support uninterrupted periodic sampling[1].

The wireless communication interface is based on the ESP32's integrated Wi-Fi module, which enables direct data transmission from sensor nodes to the cloud backend. The system must ensure reliable Wi-Fi connectivity, appropriate network configuration and secure data exchange using standard IP-based communication protocols. Variations in signal strength or temporary connectivity interruptions in agricultural environments must be handled through retry and buffering mechanisms at the device level to maintain data integrity.

Power interfaces are also fundamental. Each sensor node must operate reliably on 3.7 V Li-Ion or Li-Po battery systems. The hardware must support energy-saving modes and deep-sleep states. All components must be physically robust and weather-resistant to withstand outdoor conditions. The complete hardware interface architecture must guarantee stable electrical operation, reliable wireless communication and long-term durability.

3.1.2. Communication Requirements

The communication infrastructure of the TARAS system is designed as a scalable and reliable architecture that supports continuous IoT telemetry streaming, real-time ML inferences, disease detection and mobile app interactions. At the field level, distributed ESP32-C6 sensor nodes collect soil-moisture and environmental data from connected sensors and transmit the telemetry data directly to cloud-based services via Wi-Fi. This direct communication approach reduces system complexity while enabling timely data delivery for analysis and decision support.

The system's data infrastructure is built around a cloud-hosted PostgreSQL database, deployed on a cloud platform such as AWS. This backend architecture supports reliable data storage, efficient querying of historical records and real-time access for analytical services[3].

TARAS's backend architecture will consist of two layers: the cloud-based main backend and the service layer that manages mobile clients. The main backend consists of API services running on a commercial cloud hosting service such as AWS. Sensor data from the nodes first reaches the API layer, where it undergoes formatting before being transferred to time series tables. The backend service provides all the necessary API endpoints for users for them to view the digital twin map, receive irrigation recommendations and upload images for disease detection. This layer will include "caching" strategies to ensure devices can operate in intermittent internet conditions by keeping any recent data that has failed to transmit. Authentication will be provided via industry standard solutions such as AWS Cognito, which ensures two-way security[4]. The data model includes a structured schema consisting of sensor nodes, time-series telemetry records, field regions, digital twin snapshots, irrigation recommendation processing history, disease detection outputs and user profiles. All tables are designed to maintain ACID integrity and relationship constraints, identification mechanisms and data validation rules will be enforced. Image files used for disease detection will be stored in cloud storage, not directly in the database; only the file path and classification outputs will be stored in the database.

The system relies on the integrated Wi-Fi capabilities of ESP32 sensor nodes for wireless communication with the cloud backend[1]. Each node transmits collected soil-moisture and environmental telemetry directly over IP-based protocols, enabling reliable data delivery without the need for long-range radio transceivers or intermediate gateway hardware. This design simplifies the communication stack while maintaining sufficient bandwidth for periodic sensor updates and system monitoring.

To ensure continuity and data integrity in rural agricultural environments, where intermittent internet connectivity may occur, the system implements device-level retry and buffering mechanisms. Sensor nodes temporarily store measurements in local memory when transmission attempts fail and automatically resend buffered data once connectivity is restored. This approach ensures eventual consistency and prevents permanent telemetry loss without introducing additional gateway complexity.

Communication between the sensor nodes and the backend services is performed using secure HTTP APIs over Wi-Fi. Data packets are transmitted through encrypted channels such as HTTPS, ensuring confidentiality and integrity during transmission. On the cloud side, received telemetry is validated, stored and made available to analytics, ML and visualization services. This communication architecture provides a robust, secure and scalable end-to-end data flow from field-level sensors to cloud intelligence.

3.1.3. System and Backend Requirements

The TARAS system's data infrastructure is built on a scalable and reliable architecture designed to support continuous IoT telemetry streaming, real-time ML inferences, a disease detection module and mobile app interactions. The system's core database will be implemented using PostgreSQL, providing a robust relational foundation for structured agricultural data management[3].

To efficiently handle sensor telemetry, the architecture supports the use of the TimescaleDB extension, which may be enabled to improve performance for time-based queries, historical analysis and aggregation-heavy workloads[5]. When utilized, this extension facilitates efficient storage, partitioning and compression of sensor data. When spatial field analysis is required, the PostGIS extension can be enabled to support geospatial queries and region-based operations, ensuring seamless integration with the digital twin component[6].

TARAS's backend architecture consists of two logical layers: an AWS-based main backend and a service layer responsible for managing mobile client interactions. The main backend includes API services deployed on AWS and a PostgreSQL database hosted on AWS. Sensor data collected from field gateways is first received by the API layer, where it undergoes validation and formatting before being persisted in the database.

The TARAS backend architecture will utilize Node.js as the server-side runtime environment for managing RESTful APIs, processing

incoming sensor data and coordinating interactions between the mobile application, database services and ML modules. The backend exposes RESTful API endpoints that allow users to access digital twin maps, receive irrigation recommendations, upload plant images for disease detection and receive system notifications.

3.1.4. Software Interface Requirements

The TARAS software architecture relies on a set of coordinated interfaces that connect the ML components, backend services and mobile app. These interfaces ensure that irrigation prediction, disease diagnosis and the LLM-based advisory system operate consistently. The system's ML modules communicate with the backend through well-defined API contracts, enabling the exchange of structured sensor data, user input and model outputs.

The irrigation-prediction module operates through a time-series interface with the cloud database. The backend supplies the decision engine with aggregated historical sensor data, including soil moisture trends, environmental conditions and past irrigation records. Based on these inputs, the model produces a recommendation object that specifies whether irrigation is required, along with suggested timing and duration parameters and supporting indicators used for explainability.

The disease-detection component interacts with the system through an image-processing interface exposed by the backend. The mobile app uploads leaf images in compressed JPEG/PNG format and the backend forwards these images to the CNN inference engine. The model returns a classification label, confidence score and an explanation map if available.

The LLM-based advisory module operates on top of the system's analytical outputs and requires a specialized interface to retrieve the latest sensor data, irrigation predictions and disease-diagnosis results. The backend provides the model with a context package containing environmental data, recent recommendations and user-submitted information such as irrigation history. The LLM returns a natural-language advisory response, including explanations, warnings or scenario-based comparisons. This interface allows the mobile app to request advice in real time while ensuring that the LLM's outputs remain consistent with the most recent sensor readings and model predictions.

The mobile interface uses the designated API endpoints to retrieve ML results, send image data and request advisory messages. All interfaces must maintain compatibility with the PostgreSQL-backed data layer and must handle synchronization between the nodes, cloud and

mobile app without data loss. These software interfaces also ensure that the ML-driven components such as disease detection and LLM, integrate seamlessly with the user-facing mobile app and the backend.

3.1.5. User Interface Requirements

The TARAS mobile application serves as the primary interaction layer between the farmer and the underlying sensing, prediction and visualization ecosystem, requiring a cohesive and intuitive interface that enables efficient decision-making under varying environmental conditions. Overall, the user interface must maintain responsiveness across diverse screen sizes, uphold visual consistency aligned with the system's design identity and model outputs into actionable, farmer-friendly insights.

The user interface is structured around a persistent navigation bar that provides direct switching between tabs that will enable the user to access their desired function such as Overview, Timetable, Disease Detection and Settings. A global floating button remains visible across these primary screens to engage the AI assistant overlay. This conversational interface processes natural language queries and enables users to execute "What-if" scenario simulations.

Access to the application is managed through a role-based login interface that authenticates distinct user types, including farmers and stakeholders and adjusting interface permissions and data visibility.

The central dashboard functions as a digital twin, rendering an interactive 2D/3D visualization of the field. This model maps real-time sensor inputs, specifically soil moisture and temperature, to color-coded zones within the canvas. Users navigate this digital representation by dragging to rotate the perspective and tapping specific sectors to isolate detailed data. To support immediate decision-making, the visualization highlights zones where readings fall outside optimal ranges, providing visual alerts for conditions such as drought stress or over-irrigation.

The disease detection module utilizes a live camera viewfinder and image upload, enabling users to capture plant imagery for on-screen classification of disease types and severity. Additionally, the specific timetable interface presents a schedule of irrigation and fertilization tasks, while also providing input fields for logging fuel, energy and fertilizer consumption to support carbon footprint calculations.

Navigation shall remain simple and consistent across Android and iOS devices, using a navigation bar to provide fast access to primary screens such as dashboard, digital twin, disease detection and settings.

3.2. Detailed Description of Functional Requirements

3.2.1.1. Microcontroller - Sensor Interface

Name	**Microcontroller - Sensor Interface**
Purpose / Description	The sensor interface enables reliable acquisition of soil moisture, temperature, humidity and environmental data from IoT sensor nodes. Each ESP32-C6 node reads SHT31 sensors via I²C and the soil-moisture probe via ADC. The interface ensures proper electrical levels (3.0–3.6 V), periodic sampling and noise-resistant communication.
Inputs	● I²C readings from SHT31 temperature-humidity sensor ● ADC readings from capacitive soil-moisture sensor ● I²C/ADC configuration parameters (address, bus speed, ADC resolution, sampling rate) ● Regulated 3.0–3.6 V power input
Processing	1. Initialize I²C bus and ADC at boot 2. Perform periodic sampling of all sensors 3. Verify sensor presence via I²C acknowledgment 4. Apply calibration and scaling to raw readings 5. Filter noisy values using smoothing/averaging 6. Package processed data into Wi-Fi-ready telemetry messages 7. Timestamp each measurement for cloud synchronization
Outputs	● Validated sensor data (soil moisture, humidity and etc) ● Structured telemetry payloads for Wi-Fi transmission to the cloud backend ● Sensor status indicators (active, missing, calibrated)
Error Handling	● Retry I²C communication on missing acknowledgment ● Mark and suppress physically invalid readings ● Reset I²C bus after collisions or noise ● Suspend sampling under low-voltage conditions ● Log diagnostic flags locally for cloud-side monitoring and debugging

- **Gateway Hardware Interface Functional Requirement**

Name	**Gateway Hardware Interface**
Purpose / Description	The sensor node communication interface enables reliable transmission of processed sensor data from ESP32 nodes directly to the cloud backend via Wi-Fi. This interface is responsible for network initialization, secure data forwarding and temporary buffering to ensure continuity under intermittent connectivity conditions.
Inputs	● Structured sensor telemetry generated by the sensor interface ● Wi-Fi network credentials and configuration parameters ● System time for timestamping ● Device memory for temporary buffering
Processing	1. Initialize Wi-Fi connectivity at startup 2. Establish secure communication channels with cloud services 3. Package sensor data into standardized messages (e.g., JSON) 4. Timestamp and format data for cloud ingestion 5. Transmit telemetry via HTTP or MQTT over Wi-Fi 6. Buffer data locally during connectivity loss and retry transmission upon reconnection
Outputs	● Cloud-delivered sensor telemetry ● Transmission status indicators (success, retry, failure) ● Diagnostic logs for connectivity and transmission events
Error Handling	● Retry Wi-Fi connection on initialization or runtime failure ● Queue telemetry locally when transmission fails ● Discard or flag stale data after configurable retry limits ● Reset network interface if connectivity becomes unresponsive ● Report persistent failures to cloud-side monitoring services

- **Power Interface Functional Requirement**

Name	**Power Interface**
Purpose / Description	The power interface ensures stable and efficient energy delivery to ESP32 sensor nodes and connected sensors using a 3.7 V Li-Ion/Li-Po battery regulated to safe operating levels. It supports low-power modes, deep-sleep operation and voltage stability to maximize battery lifetime. The interface also prevents undervoltage, brownout condition, and power-related data loss.
Inputs	● 3.7 V Li-Ion / Li-Po battery input ● Regulated 3.0–3.6 V output for ESP32-C6 and sensors ● Power-management configuration (sleep intervals, wake-up triggers) ● Voltage and current readings from onboard regulators
Processing	1. Regulate battery voltage to component-safe operating levels 2. Manage deep-sleep and low-power operational modes of the ESP32 3. Monitor voltage stability to detect undervoltage and brownout conditions 4. Allocate power to sensors and communication subsystems based on sensing and Wi-Fi transmission cycles 5. Log power-related events (sleep, wake-up, low-voltage warnings)
Outputs	● Stable power delivery to the microcontroller and sensors ● Power-state indicators (active, sleep, low voltage) ● Logged power-performance data for system diagnostics
Error Handling	● Enter safe low-power mode under undervoltage conditions ● Suspend sensing and wireless transmission when power becomes unstable ● Trigger automatic system restart after a detected brownout event ● Log critical power faults for cloud-side monitoring and analysis

- **Environmental / Enclosure Interface Functional Requirement**

Name	**Environmental / Enclosure**
Purpose / Description	The environmental interface ensures that all hardware components operate reliably in outdoor conditions by providing protection against moisture, dust, UV exposure, temperature variation and mechanical stress. Weather-resistant enclosures, sealed connectors and durable materials maintain sensor and node integrity during long-term deployment.
Inputs	● Outdoor environmental exposure(rain, dust, UV, heat). ● Mechanical stress from handling or field conditions. ● Moisture and particulate ingress risks. ● Thermal load from sun exposure.
Processing	1. Seal enclosures to prevent water and dust ingress. 2. Regulate airflow or thermal dissipation to maintain safe operating temperature. 3. Shield sensors and electronics using protective casing and strain-relieved cabling. 4. Prevent corrosion through coated PCBs or protected connectors.
Outputs	● Stable device operation under outdoor conditions. ● Protected internal electronics and sensors. ● Maintained enclosure integrity and mechanical stability. ● Environmental health status indicators.
Error Handling	● Detects abnormal temperature/humidity inside enclosure. ● Trigger warnings or reduced operation if thermal or moisture limits are exceeded. ● Log environmental fault conditions for maintenance. ● Enter protective mode or shut down to prevent hardware damage.

- **Wi-Fi Uplink Communication – Functional Requirement**

Name	**Wi-Fi Uplink Communication**
Purpose / Description	Provides direct wireless data transmission from ESP32 sensor nodes to the cloud backend using Wi-Fi. This interface ensures reliable IP-based communication for periodic and event-driven telemetry delivery.

Name	**Wi-Fi Uplink Communication**
Inputs	● Sensor payloads generated by ESP32 sensor interface ● Wi-Fi network credentials and configuration parameters ● Node identifier and message metadata ● System time for timestamping
Processing	1. Initialize Wi-Fi interface at node startup 2. Establish secure network connection to the backend services 3. Package sensor data into standardized message formats (e.g., JSON) 4. Transmit telemetry periodically or event-driven via HTTP or MQTT 5. Monitor connection status and transmission acknowledgments 6. Buffer outgoing data locally during temporary connectivity loss
Outputs	● Successfully transmitted sensor data received by the cloud backend ● Transmission status indicators (success, retry, failure) ● Structured telemetry payloads available for storage and analytics
Error Handling	● Retry Wi-Fi connection on network or authentication failure ● Reattempt data transmission upon timeout or backend unavailability ● Temporarily buffer telemetry when transmission fails ● Discard or flag stale data after configurable retry limits ● Log communication and connectivity errors for cloud-side diagnostics

- **Store-and-Forward Communication Reliability Functional Requirement**

Name	**Store-and-Forward Communication Reliability**
Purpose / Description	Ensures reliable data delivery in environments with intermittent internet connectivity by implementing local caching at the ESP3 sensor node level. When Wi-Fi or cloud transmission fails, sensor data is temporarily stored on the device and

Name	Store-and-Forward Communication Reliability
	automatically synchronized with the cloud backend once connectivity is restored, preventing telemetry loss.
Inputs	● Sensor data awaiting transmission ● Wi-Fi connectivity status of the sensor node ● Cloud backend availability status ● Local storage resources on the ESP32 (RAM/flash) ● Packet metadata (timestamps, sequence numbers)
Processing	1. Detect failed Wi-Fi or cloud transmission attempts 2. Store unsent data in local buffers on the sensor node 3. Periodically check network availability for retransmission 4. Synchronize buffered packets in chronological order once connectivity returns 5. Verify successful cloud acknowledgment before deleting local copies 6. Filter duplicate packets and ensure correct ordering using timestamps or sequence identifiers
Outputs	● Reliable, lossless delivery of sensor telemetry to the cloud backend ● Updated synchronization logs indicating queued, transmitted, and retried packets ● Continuous data flow despite connectivity interruptions
Error Handling	● Apply retry and backoff strategies when repeated transmission failures occur ● Manage buffer overflow by discarding or overwriting the oldest data based on policy ● Log cache write/read errors for diagnostics ● Validate buffered packets before retransmission to prevent data corruption

- **Node-to-Cloud Communication – Functional Requirement**

Name	Node-to-Cloud Communication
Purpose / Description	Enables secure and reliable transmission of sensor data directly from ESP32 sensor nodes to the cloud backend using HTTP or MQTT protocols over Wi-Fi. This interface ensures encrypted

Name	Node-to-Cloud Communication
	communication, standardized payload formatting and uninterrupted data flow despite intermittent internet availability.
Inputs	● Validated sensor data generated by the ESP32 node ● Cloud endpoint URLs and authentication parameters ● Wi-Fi network connectivity status ● Protocol configuration parameters (HTTP/MQTT) ● System timestamp for message scheduling
Processing	1. Format sensor data into standardized JSON or binary payloads 2. Establish secure communication channels using TLS/HTTPS 3. Transmit data via HTTP POST/MQTT publish operations 4. Monitor backend responses (ACKs, status codes) to confirm delivery 5. Temporarily cache unsent data on the device during connectivity loss 6. Automatically retransmit buffered data once network connectivity is restored 7. Log communication events, including latency, error codes
Outputs	● Successfully transmitted sensor data stored in the cloud backend ● Diagnostic logs indicating transmission success, failures, and response codes ● Updated local buffer status reflecting synchronized and pending telemetry
Error Handling	● Retry failed HTTP or MQTT transmissions ● Cache telemetry locally when internet connectivity is unavailable ● Handle TLS or authentication failures by suspending uploads until a secure channel is re-established ● Discard/reformat invalid payloads before retransmission

- **Sensor Telemetry Storage and Management Functional Requirement**

Name	Sensor Telemetry Storage and Management
Purpose / Description	Defines the system's responsibility to receive, store, manage and aggregate high-frequency sensor telemetry data in a

Name	Sensor Telemetry Storage and Management
	time-series-oriented data infrastructure. The system shall ensure that collected sensor measurements can be reliably accessed and processed to support irrigation prediction, digital twin visualization and historical trend analysis.
Inputs	● Incoming telemetry packets from gateway: ○ soil_moisture ○ temperature ○ humidity ○ node_id ○ timestamp
Processing	1. Validate the structure and data types of incoming telemetry. 2. Automatically partition data by time and node identifier. 3. Apply compression policies to historical data chunks. 4. Generate continuous aggregates, including: ○ hourly soil moisture averages, ○ daily moisture trends, ○ temperature–humidity correlation metrics. 5. Expose optimized query interfaces for ml modules and digital twin services.
Outputs	● Telemetry records accessible through REST APIs. ● Summarized datasets for UI rendering, ML processing. ● Historical trend datasets for analytics and visualization.
Error Handling	● If a schema mismatch occurs, the telemetry record is marked as invalid and logged. ● If the database connection is unavailable, records are temporarily queued at the sensor nodes and forwarded once connectivity is restored.

- **Disease Detection Database Recording Functional Requirement**

Name	Disease Detection Database Recording
Purpose / Description	Stores the output of the disease detection module; including image path, classification, severity, confidence and timestamp, inside a structured disease results table in PostgreSQL.

Name	**Disease Detection Database Recording**
Inputs	● Classification result from CNN model ● Cloud storage URL of user-uploaded image ● User ID (farmer) ● Device and field metadata
Processing	1. Validate disease result schema. 2. Insert record into disease results table. 3. Update disease statistics for the field. 4. Trigger notification system if severe disease detected.
Outputs	● Database record (row) created. ● Disease history accessible via API. ● Push notification sent.
Error Handling	● If the database write operation fails, the system performs an automatic retry before reporting an error. ● If the classification output is missing, the system records the entry with the label “unidentified.”

- **Digital Twin Data Assembly Functional Requirement**

Name	**Digital Twin Data Assembly**
Purpose / Description	Supports the Digital Twin by managing field geometry, zone polygons and spatial queries using PostgreSQL + PostGIS.
Inputs	● Field boundary polygons ● Zone definitions (polygons) ● Sensor-node geolocation coordinates ● Latest moisture/temperature telemetry
Processing	1. Store and index spatial data 2. Map telemetry to nearest field zone 3. Generate zone-level moisture/temperature aggregates 4. Prepare structured datasets for heatmap rendering
Outputs	● Zone-level moisture distribution ● 2D/3D Digital Twin datasets ● Spatial queries for crop/zone analysis
Error Handling	● If a geometry record is invalid, it’s rejected and logged for review.

Name	Digital Twin Data Assembly
	● If telemetry data arrives without a spatial reference, it is assigned to an “unmapped” zone. ● If a spatial index becomes corrupted, the system rebuilds the index to restore proper query performance.

- **Disease Detection Functional Requirement**

Name	Disease Detection Functional Requirement
Purpose / Description	Enables users to upload plant leaf images and receive disease classification using the system’s CNN-based image-processing module. The module analyzes images, detects disease type/severity, generates confidence scores and provides recommendations.
Inputs	● User-uploaded plant leaf image. ● Optional metadata: crop type, field zone, timestamp. ● Pretrained CNN weights and preprocessing parameters. ● Stored disease classes in the database.
Processing	1. Validate image size, format, and resolution. 2. Preprocess image (resize, normalize, color correction). 3. Run CNN inference to classify disease type & severity. 4. Store classification output into PostgreSQL (disease table) with image path (cloud storage). 5. Forward result to mobile application for visualization.

Outputs	● Disease label
	● Severity score.
	● Confidence value.
	● API response to the mobile client.
	● Database entry containing:
	  ○ image_url
	  ○ classification
	  ○ confidence
	  ○ severity
	  ○ timestamp

Error Handling	● If the image format is invalid, the system returns an “unsupported file format” error.

Name	Disease Detection Functional Requirement
	● If the image file is corrupted, the system rejects the upload and records it in the logs. ● If the inference engine is unavailable, the system attempts a retry and then returns a error if the issue persists. ● If inserting the result into the database fails, the system logs the failure and returns a partial-success response to the client. ● If metadata is missing, the system proceeds using default assumptions defined by the backend.

- **Irrigation Prediction Request & Response Handling Functional Requirement**

Name	Irrigation Prediction Request & Response
Purpose / Description	Generates irrigation recommendations using a rule-based decision engine that evaluates aggregated sensor data, calibrated soil-moisture dynamics and predefined irrigation rules.
Inputs	● Soil moisture (time-series) ● Temperature and humidity ● Field metadata (crop type, soil type, irrigation method) ● Calibrated thresholds (field capacity, stress threshold) ● Historical irrigation markers

Processing	1. Retrieve latest time-series sensor data from PostgreSQL
	2. Smooth and validate soil-moisture signal
	3. Detect irrigation start and finish times using moisture-derivative rules
	4. Evaluate soil-moisture state against calibrated rule thresholds
	5. Apply irrigation rules to determine:
	  ● irrigation necessity
	  ● recommended duration
	  ● next irrigation timing
	6. Assign recommendation priority based on moisture deficit trend

Outputs	JSON recommendation object including:

Name	Irrigation Prediction Request & Response
	● irrigation decision (apply / delay / skip) ● recommended irrigation duration ● suggested next evaluation time
Error Handling	● If sensor data is partially missing, use last valid stabilized segment ● If database is unavailable, cache request ● If rule evaluation fails, notify backend and return unavailable

- **Role-Based Access Control & IU Authorization Functional Requirement**

Name	Role-Based Access Control & UI Authorization
Purpose/Description	Defines role-based access rules for the TARAS mobile and web interfaces. The system controls which screens, features and data are visible or accessible to users based on their assigned role.
Inputs	● Authenticated User Session ● User Role ● Requested UI Screen or Feature ● Navigation Events
Processing	1. Role Identification: Determine the user's role after successful authentication. 2. Access Evaluation: Check requested screen or feature against role-based permission rules. 3. UI Rendering: Display only the interface components permitted for the user's role. 4. Navigation Filtering: Hide or disable navigation items that are not allowed for the current role. 5. Session Enforcement: Enforce access rules during navigation and screen transitions.
Outputs	● Role-appropriate UI layout. ● Accessible navigation menu based on user role. ● Authorized access to data, visualizations and actions.

Name	Role-Based Access Control & UI Authorization
Error Handling	● If a user attempts to access a restricted screen, redirect to an authorized default screen. ● If role information is missing or invalid, terminate the session and require re-authentication. ● If permission rules fail to load, restrict access to read-only or safe default views.

- **Accessibility & Inclusive UI Support Functional Requirement**

Name	Accessibility & Inclusive UI Support
Purpose / Description	Defines accessibility features of the TARAS user interface to ensure that the mobile application is usable by farmers and stakeholders with varying physical, visual, and cognitive abilities. The interface prioritizes clarity, readability, and ease of interaction under field conditions.
Inputs	● System Accessibility Settings (Font Size, Contrast) ● Device Accessibility Preferences ● User Interaction Events ● UI Rendering Context
Processing	1. Font Scaling: Adapt text size according to system-level font and accessibility settings. 2. Contrast Enforcement: The system shall ensure a minimum contrast ratio of 4.5:1 between text and background for all primary information elements displayed in the interface[7]. 3. The system shall ensure that all interactive UI elements have a minimum touch target size that was stated in Web Content Accessibility Guidelines[8]. 4. Visual Simplification: The system shall limit the number of primary interactive elements displayed on a single screen to a maximum of five, prioritizing critical actions and information. 5. Consistent Feedback: The system shall provide immediate visual feedback for user interactions, including pressed, selected, loading, and error states.
Outputs	● Accessible UI layout with readable text and clear contrast. ● Enlarged and touch-friendly interactive elements. ● Simplified screens optimized for field usage.
Error Handling	● If accessibility preferences cannot be applied, load the default high-contrast and large-text layout. ● If UI rendering fails due to accessibility overrides, reset to a

Name	Accessibility & Inclusive UI Support
	safe default layout. ● If device accessibility settings change during runtime, re-render the UI to reflect updated preferences.

- **Mobile Application Framework & Navigation Functional Requirement**

Name	Mobile Application Framework & Navigation
Purpose / Description	Defines the initialization, localization and navigation structure of the TARAS application. It establishes the primary user interface shell, ensuring consistent navigation and language support across Android and iOS devices.
Inputs	● System Language Settings (Locale) ● System Theme Preference (Light/Dark) ● User Credentials (Login/Skip) ● Touch Interaction Events
Processing	1. Localization: Detect device system language on startup apply Turkish(TR) by default or English(EN) if explicitly set. 2. Theme Application: Apply the color schema (Light/Dark) based on system settings or user override. 3. Authentication Routing: Direct user to Login Screen or load main dashboard if a valid session token exists. 4. Navigation Mounting: Render the persistent bottom navigation bar (dashboard, timetable, settings). 5. Overlay Initialization: Attach the Global Floating Action Button (FAB) for the AI Assistant to the root view.
Outputs	● Localized Main UI Shell. ● Active Navigation Stack. ● Visual Theme (Colors/Fonts).
Error Handling	● If the system language is not supported, automatically fallback to Turkish. ● If a screen fails to mount, reset the navigation stack to the dashboard.

- **Carbon Footprint Calculation & Display**

Name	**Carbon Footprint Calculator**
Purpose / Description	The system shall provide a carbon footprint calculation feature that estimates the agricultural carbon dioxide emission impact of a field based on farmer-provided inputs. The calculated footprint shall be associated with the corresponding user and field and presented to the user through the mobile application.
Inputs	● Farmer responses to predefined multiple-choice questions ● Optional numeric inputs related to energy, fuel, or fertilizer usage
Processing	1. UI Form Handling ● Display a separate “Carbon Footprint” section in the mobile app. ● Collect answers via multiple-choice and numeric fields. ● Validate that required questions are answered before submission. 2. Backend Validation ● Validate schema (required fields, allowed options, numeric ranges). ● Map multiple-choice selections to internal coefficients. 3. CO_2e Calculation ● Compute component emissions (example components): ○ Irrigation energy emissions ○ Fertilizer-related emissions ● And then produce: ○ Total_CO2e (kg CO_2e) ○ Breakdown 4. Store a record in carbon_footprint_results table, which possible row such as: ● user_id, total_co2e, breakdown_json, timestamp 5. Return computed values to UI in a standardized JSON object for immediate display.
Outputs	● Total carbon footprint estimate (kg CO_2e) ● Component breakdown
Error Handling	● If some multiple-choice answers are missing, the UI prevents submission and highlights incomplete fields. ● If numeric inputs are out of range or invalid (negative, non-numeric), the backend rejects request with message.

Name	Carbon Footprint Calculator
	● If emission-factor mapping fails, the backend defaults that component to unavailable and flags the response with a warning. ● If database write fails, the system retries automatically; on repeated failure, it returns the computed result to the user but marks it as “not saved” and logs it for admin review.

3.3. Data Preprocessing

3.3.1. Irrigation Data Preprocessing

3.3.1.1. Purpose and Data Sources

The irrigation recommendation module relies on consistent and time-aligned field and weather data to compute crop evapotranspiration (ETc), which represents the actual water consumption of a specific crop under current atmospheric conditions and growth stage

ETc is derived by adjusting reference evapotranspiration (ET_0)—the atmospheric water demand—using the crop coefficient (Kc), which reflects crop type and growth-stage characteristics.

Based on ETc-derived water demand and observed soil moisture dynamics, the system estimates both when irrigation should be initiated (e.g., after a certain level of water depletion or time interval) and how long irrigation should be applied, converting required water depth into pump operation time through irrigation gain parameters. To support this estimation process, the system shall execute a structured preprocessing pipeline on the following inputs before any ET_0/Kc-based calculation:

- **Weather / Meteorological Data** (for ET_0 or ET_0 drivers): temperature, relative humidity, wind speed(or alternatives).
- **Field Sensor Data**: soil moisture (SM), air temperature/humidity.
- **Crop Metadata**: crop type and crop coefficient (Kc) profile parameters.
- **Irrigation Records**: irrigation events and pump duration/flow estimations for calibration and validation.

3.3.1.2. Timestamp Standardization and Time Alignment

To ensure consistent irrigation calculations based on reference evapotranspiration (ET_0), which represents the atmospheric water demand, and the crop coefficient (Kc), which adjusts ET_0

according to crop growth stage, the system shall convert all timestamps to a unified timezone and format. Sensor and weather data streams shall be aligned on a common timeline using a fixed resampling interval (e.g., hourly or daily).

3.3.1.3. Sensor Quality Control (Noise, Drift, Spikes)

The system shall apply sensor quality controls to mitigate noise, spikes, and long-term drift in soil moisture measurements. Unrealistic spikes shall be detected and excluded, smoothing shall reduce noise without suppressing irrigation effects, and baseline drift shall be monitored to generate sensor health indicators. These controls shall be applied before missing-value handling.

3.3.1.4. Missing Data Analysis and Handling

The system shall compute missing-value ratios per feature and generate a missing-data report. For ET_0 and weather-critical variables, short gaps shall be interpolated, while extended gaps shall suspend ETc computation. For soil moisture data, short gaps shall be interpolated, and long gaps shall result in reduced confidence using weather-based fallback estimation.

3.3.1.5. Context-Aware Outlier Detection

The system shall perform context-aware outlier detection rather than generic IQR filtering. Statistical outlier detection shall be applied to meteorological variables, while soil moisture outliers shall be evaluated using irrigation-aware rules that distinguish valid irrigation-induced changes from sensor anomalies.

3.3.1.6. Crop Coefficient (Kc) Profile Preparation

The system shall generate a continuous Kc time series by loading crop-specific Kc profiles and mapping planting dates to a day-after-planting (DAP) index. If calibrated Kc parameters are available, versioned calibrated values shall be applied consistently.

3.3.1.7. Derived Features for ET_0 + Kc Irrigation Logic

The system shall compute crop evapotranspiration as $\mathbf{ETc = ET_0 \times Kc}$ and derive cumulative ETc over a configurable rolling window to support irrigation recommendation decisions.

3.3.2. Image Data Preprocessing for Disease Detection

The disease detection module relies on accurate, noise-reduced, and consistently formatted image data to ensure model stability, high inference accuracy, and reproducible visual diagnostics. To support both training and inference, the system shall apply a structured preprocessing pipeline to all plant leaf images captured/uploaded through the TARAS application. This pipeline ensures that the CNN-based disease classifier operates on standardized, clean and meaningful visual inputs.

3.3.2.1. Raw Image Validation

The system shall validate the integrity and format of each incoming image before performing any analytical operation. Validation includes checking file format (JPEG/PNG), verifying that file size does not exceed predefined limits, ensuring that the image is not corrupted, confirming minimum acceptable resolution for classification. Images failing any of these checks shall be rejected and logged for quality assurance.

3.3.2.2. Image Resizing and Aspect-Ratio Normalization

In order to maintain consistency across the CNN input pipeline, the system shall resize all images to the model's expected input dimensions, preserve aspect ratio when possible and apply center-cropping/padding to prevent distortion. This ensures that all images conform to the numerical structure required by the model.

3.3.2.3. .Color Space Standardization

To ensure consistent lighting interpretation and feature extraction, the system shall convert all images to a standardized color space. Operations include converting raw images to RGB and normalizing pixel intensity ranges to 0–1 (depending on model specification), This enables reproducible visual feature extraction across devices.

3.3.2.4. Noise Reduction

Mobile field images often contain shadows, glare or compression artifacts. The preprocessing pipeline shall apply optional filters such as Gaussian blur (low-intensity smoothing), median filtering for salt-and-pepper noise and contrast normalization for uneven lighting conditions. These operations reduce noise while preserving disease-relevant texture patterns.

3.3.2.5. Leaf Region Isolation

The system performs leaf segmentation to isolate disease-relevant regions. Methods include color-threshold masking (green-channel analysis), background removal and contour detection. Isolating the leaf region reduces false positives caused by soil, sky, or human-made background elements.

3.3.2.6. Data Augmentation for Model Generalization

To improve model robustness and prevent overfitting, the training pipeline shall apply controlled augmentation techniques, such as random rotations and flips, controlled brightness/contrast adjustments, and slight zoom or crop transformations. Augmentation must not distort disease-critical features.

3.3.2.7. Image Quality Assessment Checks

Before inference or training, the system shall evaluate image quality using criteria such as blur detection (Laplacian variance), lighting adequacy, obstruction analysis (e.g., partially visible leaves). Images failing minimum quality thresholds shall be flagged or optionally rejected.

3.3.2.8. Dataset Class Distribution Documentation

To support fairness and balanced classification accuracy, the system shall compute and store the distribution of disease classes in the annotated dataset. This documentation includes number of images per disease category, identification of underrepresented diseases, recommendations for synthetic augmentation or targeted data collection.

3.3.2.9. Normalization for Inference

During inference, the system shall apply the same normalization parameters used during training, specifically mean subtraction, standard deviation division, reordering to the model's expected tensor structure. Consistency between training and inference pipelines is mandatory for reliable performance.

3.4. Non-Functional Requirements

3.4.1. Performance Requirements

Under normal operating conditions, sensor data is expected to become available to the backend and mobile application within **30 seconds to 1**

minute after collection. The mobile application is expected to respond to user interactions such as tab navigation, data loading and dashboard updates within **1–2 seconds**. Irrigation analysis and recommendation generation is expected to complete within a few seconds after relevant data becomes available. Disease-detection results are expected to be delivered within **5–10 seconds** following image upload, depending on network conditions. In cases of limited or unstable connectivity, the system is expected to immediately display cached or last-known data to maintain usability.

3.4.2. Reliability Requirements

The system is expected to work reliably in real agricultural environments where network interruptions and temporary component issues may occur. Under normal conditions, sensor data is expected to be successfully delivered for around **95–98%** of the time and any missing data being limited to short gaps. If communication is temporarily unavailable, the system is expected to continue using the last known data for up to **10–30 minutes** until new measurements arrive. Once connectivity is restored, data synchronization between field devices and the backend is expected to resume automatically within a few minutes. Any data loss is expected to be minimal and occasional rather than continuous. In case of temporary failures in sensor nodes, gateways or backend services, the system is expected to keep operating in a reduced but stable state and return to normal operation automatically when the issue is resolved.

3.4.3. Availability Requirements

The system is expected to remain available for regular use throughout the agricultural season. Backend services are expected to achieve an overall availability of around **97–99%** under normal conditions. The application is expected to be accessible whenever the user's device is operational and core features remain available even during temporary outages. Planned maintenance or short service interruptions are expected to be infrequent and limited in duration, usually not exceeding a few minutes at a time. In the event of connectivity loss, the system is expected to continue providing access to previously synchronized data. Once connectivity is restored, the system is expected to resume normal operation automatically without requiring user intervention.

3.4.4. Scalability Requirements

The system is expected to scale gradually as the number of fields and deployed sensor nodes increases over time. In early deployments, the platform will start with a prototype setup in a limited field area, initially using two sensor nodes. The number of sensors is not fixed and is expected to be adjusted based on collected data, performance and the

quality of recommendations produced by the system. Data storage and processing components are expected to accommodate continuous growth across multiple growing seasons. Therefore, the overall data volume is expected to increase steadily as new measurements and field records are accumulated, rather than increasing abruptly.

3.4.5. Security Requirements

The system is expected to protect user accounts and field related data from unauthorized access. Under normal operating conditions, only authenticated users shall be able to access the system. And each user is supposed to only see the information that they are allowed. User login information and other sensitive data are expected to be protected during both transmission and storage. Data related to fields, sensor outputs and analysis results are expected to be available only to the authorized users. Images and inputs uploaded by the users and inputs are expected to be secure in order to prevent data exposure. As the number of users increase, the system is expected to maintain this level of security without affecting usability and performance.

3.4.6. Usability Requirements

The system is expected to be usable by farmers with limited technical experience. Under normal operating conditions, a first time user is expected to understand and perform basic tasks like viewing field data or checking irrigation recommendations. Users should access most actions in the mobile app within 2 to 4 steps. Visual elements are expected to be readable under outdoor conditions and key information shall be distinguishable within three to five seconds of viewing. New users are expected to become familiar with the system's main features after two short tutorial sessions at most. As the system evolves, the user effort required to perform common tasks is expected to remain in the limits stated previously in this paragraph.

3.4.7. Maintainability Requirements

The system is expected to be maintainable with limited effort. Under normal operating conditions, common maintenance activities like configuration changes, minor bug fixes, or small updates are expected to be completed within 1 to 2 maintenance sessions with 10 to 30 minutes of down time at most. Software updates are supposed to be able to be deployed remotely. Also they shall not require more than one technician. When a system fault occurs, the cause of the problem should be identified in 30 to 60 minutes. Minor fixes are expected to be resolved in a work day, while more complex issues are expected to be resolved in 2 to 5 work days.

3.4.8. Portability Requirements

The system is supposed to be portable across commonly used platforms and environments. Under normal operating conditions, the mobile app is expected to run on both iOS and Android devices without platform specific changes. Backend components are expected to be deployable across different server and cloud environments with one configuration adjustment. System migration to a new environment is expected to be completed in one work day without affecting stored data or functionality.

4. Analysis - UML

4.1. Use Cases

4.1.1. Use Case Diagram

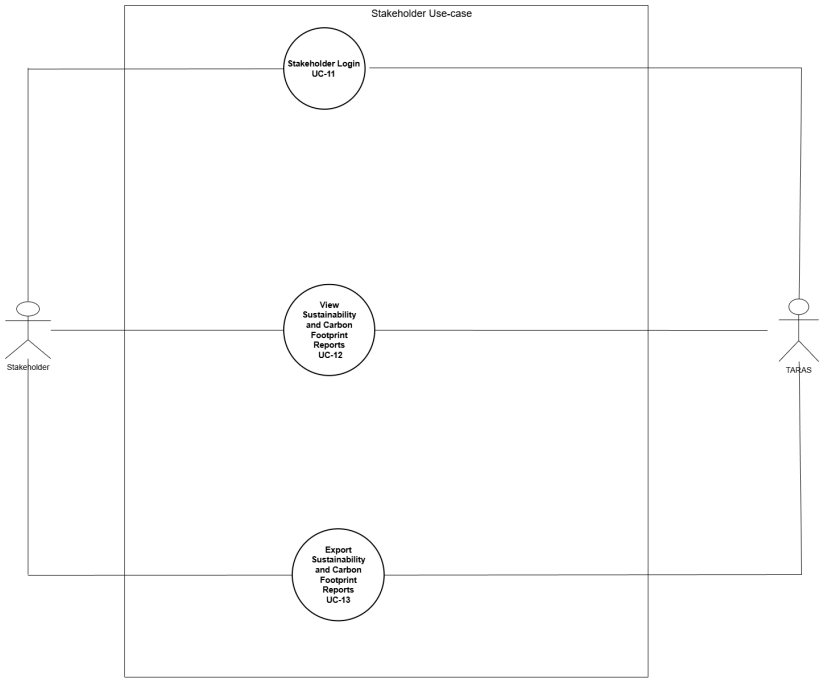


Diagram 1: Stakeholder Use Case Diagram

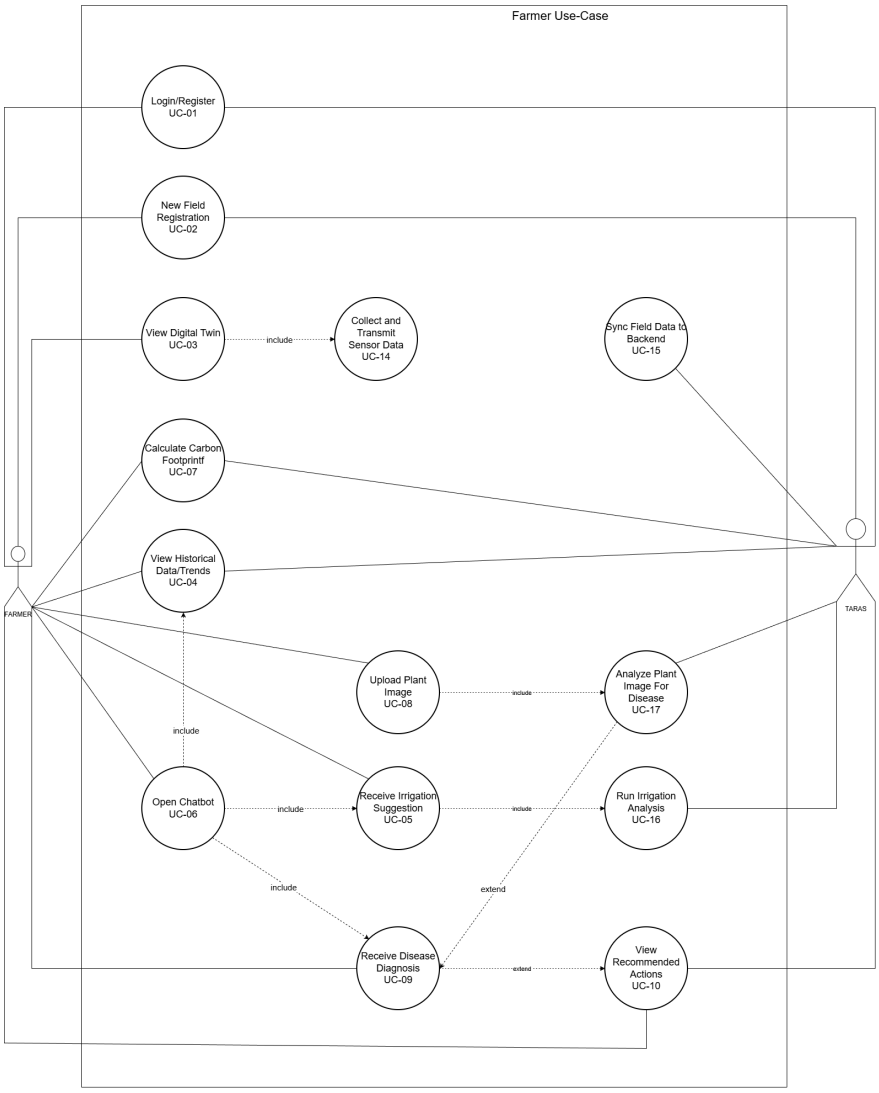


Diagram 2: Farmer Use Case Diagram

4.1.2. Description of Use Cases

Use Case Name	**Login/Register**
Use Case #	UC-01
Actors	Farmer
Description	The system manages user login and registration and grants access to the TARAS mobile application according to the user's role.
Precondition	● The user has access to the TARAS application installed.

	● The user has a stable internet connection for the login/registration process. ● For login, the user already has a registered account.

Scenario	1. The system displays a login/register screen.
	2. User chooses Login or Registration
	3. For login:
	  a. User inputs credentials
	  b. System verifies information
	4. For registration:
	  a. User inputs required information
	  b. The system validates the inputs and creates a new user account.
	5. Upon successful authentication, the system loads the appropriate dashboard

Postcondition	● Users are authenticated and granted access to the system according to their role. ● The user session is initialized and stored for further operations.
Exceptions	● Incorrect username/password = System displays an error message. ● Weak or missing registration information = System rejects input and prompts correction. ● Network failure = System cannot authenticate and inform the user. ● Suspended or disabled account = System denies access.
Related Use Cases	

Use Case Name	**New Field Registration**
Use Case #	UC-02
Actors	Farmer
Description	The system records a new field for a registered farmer and links it to the selected sensor nodes so that incoming data can be associated with the correct field.
Precondition	● The farmer is already logged into the system. ● Sensor nodes are already in the field.
Scenario	● System scans for nearby sensor nodes/shows available node IDs. ● Farmer selects which sensor nodes belong to this field.

	● The system links the selected node IDs to the new field. ● The system confirms that the field has been registered.
Postcondition	● A field record is created in the system. ● The selected sensor nodes are associated with this field, allowing incoming data to be grouped accordingly.
Exceptions	● No sensor nodes detected = System notifies the farmer. ● Invalid or duplicate IDs = System asks for correction. ● Network issues = System cannot complete the registration.
Related Use Cases	

Use Case Name	**View Digital Twin**
Use Case #	UC-03
Actors	Farmer
Description	Users view the real-time and historical moisture/temperature distribution of the field via digital twin.
Precondition	Sensor data exists in DB.
Scenario	1. The farmer opens the digital twin page. 2. The app sends requests for the latest sensor values. 3. Backend queries DB hypertables. 4. Aggregated & real-time values returned. 5. Heatmap rendered on device.
Postcondition	Digital Twin updated with fresh data.
Exceptions	● No recent sensor data available. ● The backend cannot access DB. ● Device offline = cached data shown.
Related Use Cases	UC-14

Use Case Name	View Historical Data/Trends
Use Case #	UC-04
Actors	Farmer
Description	The system displays historical information for a selected field over a chosen time range, helping the farmer understand past field conditions and the effects of irrigation decisions.
Precondition	● Farmer is logged in. ● At least one field is registered and linked to sensor nodes. ● Historical data exists in the system for the selected field (or at least some past measurements are stored).
Scenario	1. Farmer chooses the history/trends option in the app. 2. System displays a time-range selector 3. Farmer selects a time range and confirms. 4. The system retrieves historical sensor and/or irrigation data for that period. 5. The system displays graphs or charts (e.g., moisture vs. time, temperature vs. time, irrigation events on the timeline). 6. Farmers may scroll, zoom, or switch between variables (moisture, temperature, irrigation). 7. Farmer closes the view or returns to the field overview.
Postcondition	● The farmer has viewed historical trends for the selected field over the chosen period.
Exceptions	● No data available for the selected period = System shows an informative “No data for this period” message and suggests another range. ● Partial data gaps = System shows available data and may mark missing intervals. ● Network/connectivity issues = System cannot fetch new history and offers cached trends if available.
Related Use Cases	UC-06

Use Case Name	Receive Irrigation Suggestion
Use Case #	UC-05
Actors	Farmer
Description	The system provides an irrigation recommendation for the selected field based on previously computed irrigation analysis results, including ETc-based water demand and soil moisture dynamics. The recommendation indicates when irrigation should be initiated and how long irrigation should be applied, presented in a clear and farmer-friendly format.
Precondition	● The farmer is logged into the system. ● At least one field is registered and selected. ● Irrigation analysis results are available for the field.
Scenario	1. The farmer opens the field dashboard in the mobile application. 2. The system checks whether a valid and up-to-date irrigation analysis result is available for the selected field. 3. If available, the system retrieves the most recent irrigation analysis outputs, including irrigation timing indicators and estimated irrigation duration. 4. The system interprets these outputs and converts them into a farmer-oriented recommendation format (e.g., textual guidance, highlighted indicators, or visual cues). 5. The recommendation is displayed on the dashboard, clearly indicating when irrigation should start and how long it should be applied. 6. The farmer reviews the recommendation and decides whether to act on it.
Postcondition	● The irrigation recommendation is presented to the farmer for decision-making. ● The recommendation remains accessible until a new analysis result replaces it.
Exceptions	● No irrigation analysis result available: The system informs the farmer that a recommendation cannot currently be provided. ● Network or backend failure: The system displays the last successfully retrieved recommendation, if available.
Related Use Cases	UC-06,UC-16

Use Case Name	AI Decision-Support Chatbot
Use Case #	UC-06
Actors	Farmer
Description	The farmer interacts with a conversational AI assistant that uses a Large Language Model (LLM) to interpret sensor data, irrigation recommendations, historical trends and disease-detection results. The chatbot provides natural-language explanations guidance, and actionable decision-support suggestions tailored to the field's current conditions.
Precondition	● The user is authenticated and has access to the chatbot interface. ● Recent sensor data, irrigation recommendations, or disease analysis results are available. ● The LLM service is online and reachable by the backend.
Scenario	1. The user opens the AI assistant interface within the mobile application. 2. The user enters a question, command or request for advice (e.g., *"Should I irrigate today?"* or *"What does this disease mean for my field?"*). 3. The system retrieves relevant contextual data: ● Latest sensor readings (soil moisture, temperature, humidity) ● Irrigation recommendations generated by the recommendation service. ● Disease-detection results, if available ● Weather and historical trends (optional) 4. The backend compiles this data into a structured context package. 5. The LLM processes the user's question together with the contextual data. 6. The LLM generates a natural-language response with explanations or recommended actions. 7. The system returns the response to the user through the chat interface. 8. The user reads or follows the provided guidance.
Postcondition	● The farmer receives a personalized, context-aware advisory message. ● The conversation may be logged for future analytics or model improvement.
Exceptions	● The LLM service is unreachable or times out. ● No relevant sensor or irrigation data is available for the query. ● User query is ambiguous or cannot be interpreted. ● Backend fails to compile context data. ● Response generation error occurs (model failure).
Related Use	UC-04, UC-05, UC-09

Use Case Name	AI Decision-Support Chatbot
Cases	

Use Case Name	Calculate Carbon Footprint
Use Case #	UC-07
Actors	Farmer
Description	The farmer requests a carbon footprint calculation for a selected field by providing basic activity-related inputs. The system uses this information together with existing field data to estimate the environmental impact and presents the result to the farmer [9].
Precondition	● Farmer is logged in. ● At least one field is registered and selected. ● The farmer has access to the carbon footprint feature.
Scenario	1. Farmer opens the “Carbon Footprint” or “Environmental Impact” section. 2. The system prompts the farmer to enter required inputs (e.g., irrigation method, energy source, usage level). 3. Farmer provides the requested inputs and confirms the calculation request. 4. The system processes the provided information together with existing field data. 5. The system calculates the estimated carbon footprint for the field. 6. The calculated result is displayed to the farmer in a clear and understandable format.
Postcondition	● A carbon footprint estimate is generated for the selected field. ● The result becomes available for viewing and comparison over time.
Exceptions	● Required inputs are missing = System asks the farmer to complete the information. ● Calculation cannot be completed = System informs the farmer that the result is temporarily unavailable.
Related Use Cases	

Use Case Name	Upload Plant Image
Use Case #	UC-08
Actors	Farmer
Description	The user selects or captures a plant leaf image and uploads it to the system for disease analysis.
Precondition	● The user is authenticated. ● The device has network connectivity/queued upload mode enabled.
Scenario	1. The user navigates to the disease detection page. 2. The user captures an image or selects an existing image. 3. The system validates the selected image type and size. 4. The image is uploaded to the backend for further processing. 5. The system acknowledges successful upload.
Postcondition	The image is received and stored temporarily by the backend for analysis.
Exceptions	● Invalid or unsupported file format. ● Image corrupted or unreadable. ● Network failure during upload. ● Upload exceeds size limit.
Related Use Cases	UC-17

Use Case Name	Receive Disease Diagnosis
Use Case #	UC-09
Actors	Farmer
Description	After uploading an image, the user receives the disease classification, confidence score and severity returned by the system.
Precondition	● A valid image has been uploaded. ● The inference engine is available.
Scenario	1. The system preprocesses the uploaded image. 2. The system runs the disease-detection model on the image.

Use Case Name	Receive Disease Diagnosis
	3. The model generates a label, severity level and confidence score. 4. The backend sends the diagnosis result to the user's device. 5. The user views the diagnosis.
Postcondition	● Diagnosis is displayed to the user. ● The result is stored in the database
Exceptions	● Model service unavailable. ● Inference timeout. ● Disease cannot be identified with sufficient confidence. ● Backend failure returning the result.
Related Use Cases	UC-06,UC-10, UC-17

Use Case Name	View Recommended Actions
Use Case #	UC-10
Actors	Farmer
Description	The user views actionable recommendations generated based on the diagnosed disease (e.g., treatment steps, risk warnings).
Precondition	● A diagnosis has been successfully generated ● Recommendation rules or mappings exist for the detected disease.
Scenario	1. The system matches the diagnosed disease to predefined recommendation rules. 2. The system generates appropriate treatment or monitoring actions. 3. The system presents the recommended actions on the app. 4. The user reviews the suggested next steps.
Postcondition	● Users receive actionable treatment guidance.
Exceptions	● No recommendation exists for the specific disease. ● Backend fails to retrieve recommendation rules. ● User interface fails to load the recommendation section.
Related Use	UC-09

Use Case Name	View Recommended Actions
Cases	

Use Case Name	Stakeholder Login
Use Case #	UC-11
Actors	Stakeholder
Description	A stakeholder logs into the platform to access high-level summaries, sustainability metrics and environmental reports for the agricultural fields they have a stake in.
Precondition	● The stakeholder has a valid account and credentials. ● System authentication services are available.
Scenario	1. Stakeholder opens the dashboard login page. 2. Stakeholder enters email/username and password. 3. System verifies credentials. 4. The system loads the stakeholder dashboard with authorized field data. 5. Stakeholders gain access to summary and reporting tools.
Postcondition	● Stakeholders are successfully authenticated and connected to their dashboard.
Exceptions	● Invalid credentials. ● User account disabled or missing necessary permissions. ● Authentication service unavailable. ● Network failure.
Related Use Cases	

Use Case Name	View Sustainability and Carbon Footprint Reports
Use Case #	UC-12
Actors	Stakeholder
Description	The stakeholder views sustainability and carbon footprint for a selected time period to support monitoring and reporting.
Precondition	● Stakeholder is authenticated and logged into the system. ● Relevant historical field, irrigation, and sustainability data exists in the database.
Scenario	1. The stakeholder selects a time period (year, month, quarter, or custom range). 2. The stakeholder selects the report type: a. Field Summary b. Sustainability Indicators c. Irrigation Efficiency d. Carbon Footprint & Environmental Metrics 3. The system retrieves aggregated and processed data for the selected period and report type. 4. The system displays the report using charts, tables, and key performance indicators.
Postcondition	Selected reports are successfully displayed to the stakeholder.
Exceptions	● No data available for the selected period. ● Required metrics cannot be computed due to missing or incomplete data. ● Backend analytics or query service failure.
Related Use Cases	UC-13

Use Case Name	Export Sustainability and Carbon Footprint Reports
Use Case #	UC-13
Actors	Stakeholder
Description	The stakeholder exports sustainability and carbon footprint for a selected

Use Case Name	Export Sustainability and Carbon Footprint Reports
	period in a downloadable format for reporting.
Precondition	● Stakeholder is authenticated and logged into the system. ● Relevant historical data exists in the database. ● Report data can be generated for the selected period.
Scenario	1. The stakeholder selects a time period (year, month, quarter, or custom range). 2. The stakeholder selects the report type: a. Field Summary b. Sustainability Indicators c. Irrigation Efficiency d. Carbon Footprint & Environmental Metrics 3. The stakeholder selects an export file format (CSV or Excel). 4. The system generates the requested report file. 5. The system provides the file as a downloadable resource.
Postcondition	The selected report is successfully exported and made available for download.
Exceptions	● No data available for the selected period. ● Export generation failure. ● Backend query timeout or report rendering error.
Related Use Cases	UC-12

Use Case Name	Collect and Transmit Sensor Data
Use Case #	UC-14
Actors	System
Description	Sensor nodes deployed in the field automatically collect environmental measurements and transmit them directly to the TARAS cloud backend via Wi-Fi connectivity. The received data are stored and used for field monitoring, digital-twin updates, and irrigation decision support.
Precondition	● The sensor node is powered on and calibrated. ● The node is correctly registered in the system and associated with a

	specific field. ● Wi-Fi or alternative internet connectivity is available. ● The sampling interval is configured.
Scenario	1. The sensor node collects environmental measurements in the field. 2. The node prepares the measurements as a structured data payload. 3. The node transmits the data directly to the backend system via Wi-Fi. 4. The backend receives the data and verifies the associated field and node identity. 5. If valid, the backend stores the new measurements in the database. 6. The backend updates the field's latest status. 7. The system automatically triggers follow-up processes (digital-twin update, trend analysis, irrigation model input).
Postcondition	● New sensor data is stored in the database and available to farmer dashboards. ● The digital twin and field status are updated based on the latest measurements.
Exceptions	● If new data cannot be received due to connectivity issues, the system continues to display the most recent available data. ● The system may notify the farmer if sensor data has not been received for an unusually long period.
Related Use Cases	UC-03

Use Case Name	**Sync Field Data to Backend**
Use Case #	UC-15
Actors	System
Description	Field data collected from sensor nodes and the mobile application, including sensor readings and irrigation-related records, are synchronized with the cloud backend to ensure that all field information is safely stored and kept up to date.
Precondition	● Sensor nodes and/or the mobile application have collected new field

	data. ● At least one field is registered in the system. ● Wi-Fi or alternative internet connectivity is available at the sensor node or mobile device.
Scenario	1. New field data becomes available on a sensor node or the mobile application. 2. The system prepares the collected data for transmission to the backend. 3. When an internet connection is available, the system transmits the data directly to the backend. 4. The backend receives the data and stores it in the database. 5. The records associated with the relevant field are updated. 6. Updated information becomes available for farmer dashboards, digital-twin visualizations, historical analysis, and recommendations.
Postcondition	● Field data stored on sensor nodes or mobile devices is synchronized with the central TARAS system. ● The backend contains the latest version of field-related information.
Exceptions	● If no internet connection is available, the system temporarily stores the data locally and retries synchronization later. ● If synchronization fails repeatedly, the system may display a warning or status notification.
Related Use Cases	

Use Case Name	**Run Irrigation Analysis**
Use Case #	UC-16
Actors	System
Description	The system performs irrigation analysis for a registered field by processing sensor data, weather inputs, crop metadata, and irrigation records. The analysis produces irrigation-related indicators that are later used to generate irrigation recommendations and visual feedback.
Precondition	● A field is registered and linked to active sensor nodes. ● Sensor data and supporting inputs are available.

	● Irrigation analysis services are operational.
Scenario	1. The arrival of new sensor data, updated weather information or a scheduled evaluation triggers the irrigation analysis process. 2. The system gathers the required field-related data from the database. 3. Irrigation data preprocessing steps are executed to ensure temporal consistency and data quality. 4. The system performs ETc-based irrigation analysis using the preprocessed data. 5. Irrigation timing indicators and estimated irrigation duration parameters are derived. 6. The analysis results are stored in the backend and marked as available. 7. The system updates dependent modules.
Postcondition	● Updated irrigation analysis results are stored for the field. ● The results become available for recommendation generation and historical inspection.
Exceptions	● Insufficient or invalid data: The analysis is delayed or marked as unavailable. ● Processing error: The system logs the failure and follows the corresponding procedure.
Related Use Cases	UC-05

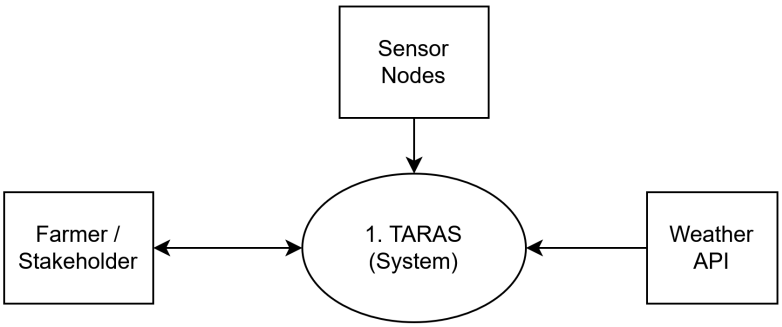
Use Case Name	**Analyze Plant Image for Diseases**
Use Case #	UC-17
Actors	System (Inference Engine, Backend Services)
Description	The system automatically processes an incoming plant image, runs the CNN model to identify potential diseases and stores the result for retrieval by the user.
Precondition	● A valid image upload request has been received from the mobile client. ● The CNN model is available and loaded on the server.
Scenario	1. The system receives the uploaded leaf image. 2. The system performs validation (format, resolution, corruption check).

Use Case Name	Analyze Plant Image for Diseases
	3. The image is preprocessed (resize, normalization). 4. The system executes CNN inference to detect disease type/severity. 5. The system generates confidence scores and optional explanation maps. 6. The system stores the classification result in the database. 7. The system returns the structured result to the mobile application.
Postcondition	● The image analysis result is stored in the disease_results table. ● The result becomes available for display in the disease detection module.
Exceptions	● Model service unavailable or inference timeout. ● Image preprocessing failure. ● Database insertion error. ● Unrecognized disease or insufficient confidence.
Related Use Cases	UC-08, UC-09

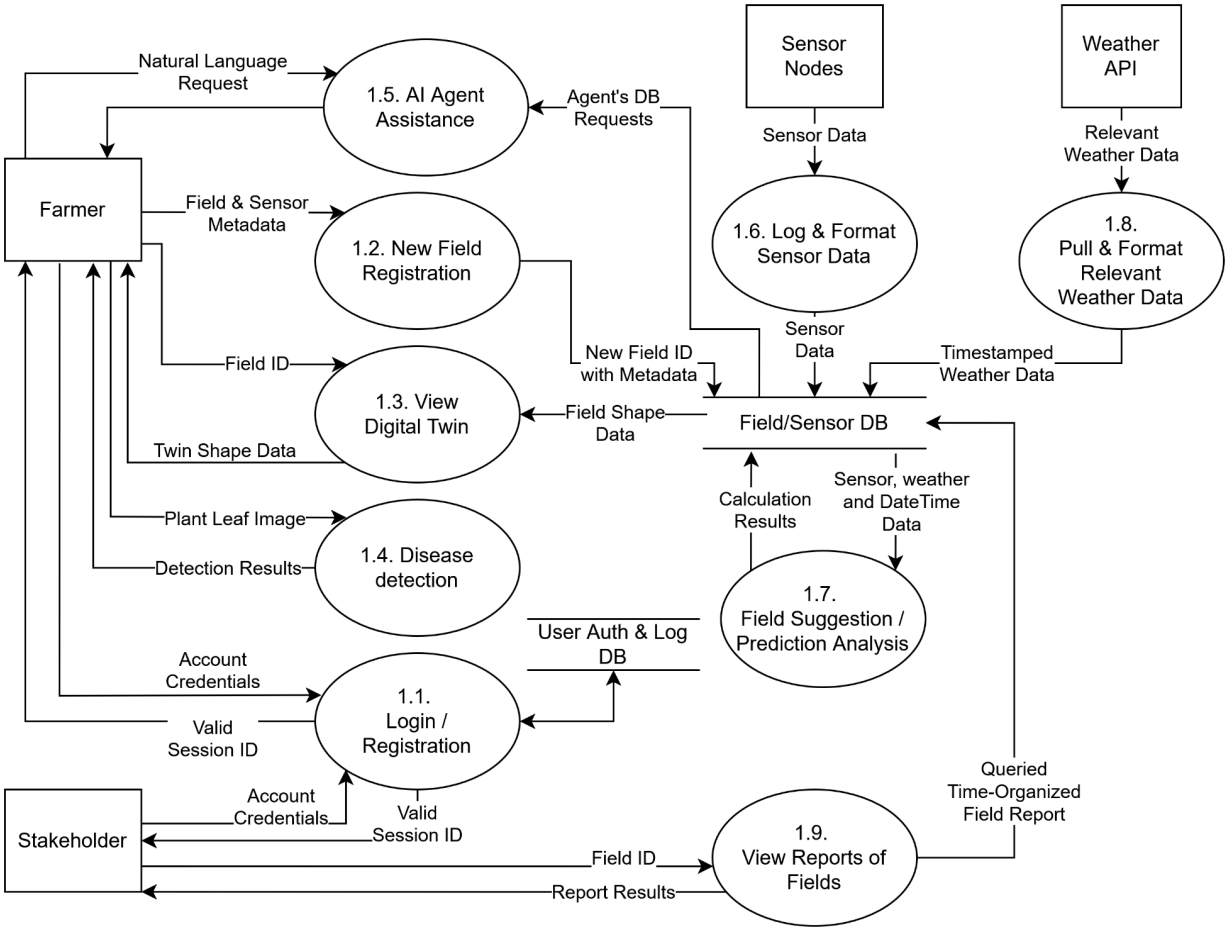
4.2. Functional Modelling(DFD)

4.2.1. DFD Diagrams

4.2.1.1. Level 0 - Context Diagram



4.2.1.2. Level 1 DFD



5. Conclusion

This Software Requirements Specification (SRS) defines the architectural and functional scope of TARAS and provides a strong foundation for system implementation. The document outlines how the platform bridges complex data analysis with practical agricultural decision-making to support irrigation and plant health management.

The proposed design integrates IoT sensing, machine-learning models, and digital-twin visualization within a unified cloud-based architecture, while addressing real-world agricultural constraints such as intermittent connectivity through buffering and synchronization mechanisms.

Overall, this SRS establishes a shared technical understanding among stakeholders and serves as a comprehensive blueprint for developing TARAS as a scalable, secure, and user-friendly precision agriculture system.

6. References

[1] Espressif Systems, “ESP32-C6 Technical Reference Manual,” 2023.
[2] “Systematic review of machine learning applications in sustainable agriculture: Insights on soil health and crop improvement,” PHYTON, vol. 94, 61375, 2025.
[3] PostgreSQL Global Development Group, PostgreSQL Documentation
[4] Amazon Web Services, Amazon Cognito – Developer Guide, AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/cognito/
[5] Timescale Inc., “TimescaleDB Documentation – Continuous Aggregates,” 2024.
[6] PostGIS Project, PostGIS Manual
[7] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1 - Success Criterion 1.4.3: Contrast (Minimum),” W3C Recommendation, 2018.
[8] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1 - Success Criterion 2.5.5: Target Size,” W3C Recommendation, 2018.
[9] ISO, ISO 14064-1: Greenhouse Gases - Carbon Footprint Quantification, International Organization for Standardization, 2018.